

MODULE 1

OBSERVABILITY & DATADOG

FUNDAMENTALS

The first principles every practitioner needs

From the three pillars to platform tour, in one volume

Observability · SLOs · Golden Signals · Platform · Cost · AI

The orientation module: vocabulary, concepts, and platform tour
for everyone joining a team that runs on Datadog

Document Version 1.0 · 2026

Table of Contents

Table of Contents	2
1. Observability vs Monitoring	5
1.1 The Core Distinction	5
1.2 A Concrete Illustration.....	5
1.3 The Three Pillars (Plus One).....	5
Metrics — the dashboards	6
Logs — the witness account	6
Traces — the journey	6
Profiles — the X-ray.....	6
1.4 How They Complement Each Other	6
1.5 Which One When?.....	7
2. SLIs, SLOs, and SLAs	8
2.1 The Three Letters.....	8
2.2 SLIs — Picking the Right Metric	8
2.3 The SLO Formula.....	8
2.4 The Magic of 9s — And the Budget They Create.....	9
2.5 Error Budgets — The Operational Genius	9
2.6 SLA — The External Contract.....	10
2.7 SLOs in Datadog.....	10
2.8 Real-World SLO Story	10
3. Golden Signals	12
3.1 The Four Signals.....	12
3.2 Latency — Percentiles, Not Averages.....	12
3.3 Latency — Success vs Failure.....	12
3.4 Traffic.....	13
3.5 Errors	13
3.6 Saturation	13
3.7 The Golden Signals in Action — One Service Dashboard	14
3.8 RED and USE — Two Subsets You'll Hear About	14
3.9 Real-World Lesson.....	14
4. Datadog Building Blocks	16
4.1 Organizations and Sites	16
4.2 Users, Roles, and RBAC.....	16
Practical RBAC patterns.....	16
4.3 API Keys and Application Keys.....	17

4.4 Integrations	17
4.5 The Datadog Agent	18
4.6 Tags	18
4.7 Unified Service Tagging	19
How to set them	19
4.8 Putting It Together	19
5. Data Flow and Cost Levers	21
5.1 The Universal Pipeline	21
5.2 The Per-Product Cost Picture	22
5.3 The Five Most Important Cost Levers	22
Lever 1 — Metric cardinality	22
Lever 2 — Log indexing vs ingestion	22
Lever 3 — APM trace sampling	22
Lever 4 — Retention duration	22
Lever 5 — Host vs container counts	23
5.4 Cost Allocation	23
5.5 A Real Cost-Optimization Story	23
5.6 Cost Discipline as a Practice	23
6. Key Platform Views	25
6.1 The View Map	25
6.2 Service Catalog	25
6.3 Infrastructure (Host & Container Map)	25
6.4 APM	26
6.5 Logs	26
6.6 Real User Monitoring (RUM)	26
6.7 Synthetics	27
6.8 Dashboards	27
6.9 Notebooks	27
6.10 The Incident-Response Tour	27
7. Unified Telemetry Model	29
7.1 The Problem It Solves	29
7.2 The Join Keys	29
7.3 The Cross-Pillar Joins in Action	29
7.4 Three Things to Get Right	30
1. Use the same env, service, version across the stack	30
2. Inject trace IDs into log records	30
3. Use consistent attribute names	30

7.5 The Failure Mode of Disunification	30
7.6 Beyond Datadog — OpenTelemetry Semantics	31
8. Datadog vs Other Observability Tools	32
8.1 The Major Players	32
8.2 Side-by-Side on Key Dimensions	32
8.3 Datadog vs Dynatrace	33
8.4 Datadog vs New Relic	33
8.5 Datadog vs Splunk	33
8.6 Datadog vs the Open-Source / OTel Stack	34
8.7 Choosing — The Decision Tree	34
9. AI Assistants in Datadog	36
9.1 The Three Things, At a Glance	36
9.2 Watchdog	36
What Watchdog catches	36
How to use Watchdog	36
Limitations	36
9.3 Anomaly and Forecast Detection	37
9.4 Bits AI	37
Things Bits AI is good at	37
Things to remember	38
9.5 Three Layers Working Together	38
9.6 The Realistic Outlook	38
10. Conclusion	39

1. Observability vs Monitoring

Every observability conversation eventually arrives at the same question: "Isn't observability just a fancy word for monitoring?" The answer is no — and the distinction matters because it changes what you build, how you instrument, and what questions you can ask of your systems. This chapter clarifies the difference in plain language, then introduces the four data types that make up modern observability: metrics, logs, traces, and profiles.

1.1 The Core Distinction

Dimension	Monitoring	Observability
Question it answers	Is the system in a known-bad state?	Can I understand any state the system is in?
Designed for	Known failure modes	Unknown failure modes
Data approach	Pre-defined metrics and dashboards	Rich, high-cardinality telemetry to query later
Mental model	Check the alarms	Investigate, ask questions, follow trails
When it shines	Steady-state operations	Novel incidents, complex distributed systems
Failure mode	Misses unanticipated issues	Expensive if instrumentation isn't thoughtful

Monitoring asks the questions you already knew to ask. Observability lets you ask new questions after something has gone wrong. A team running a monolith with 20 services can survive on monitoring; a team running 200 microservices cannot — they will encounter failure modes their dashboards were never designed for, and they need raw data rich enough to investigate after the fact.

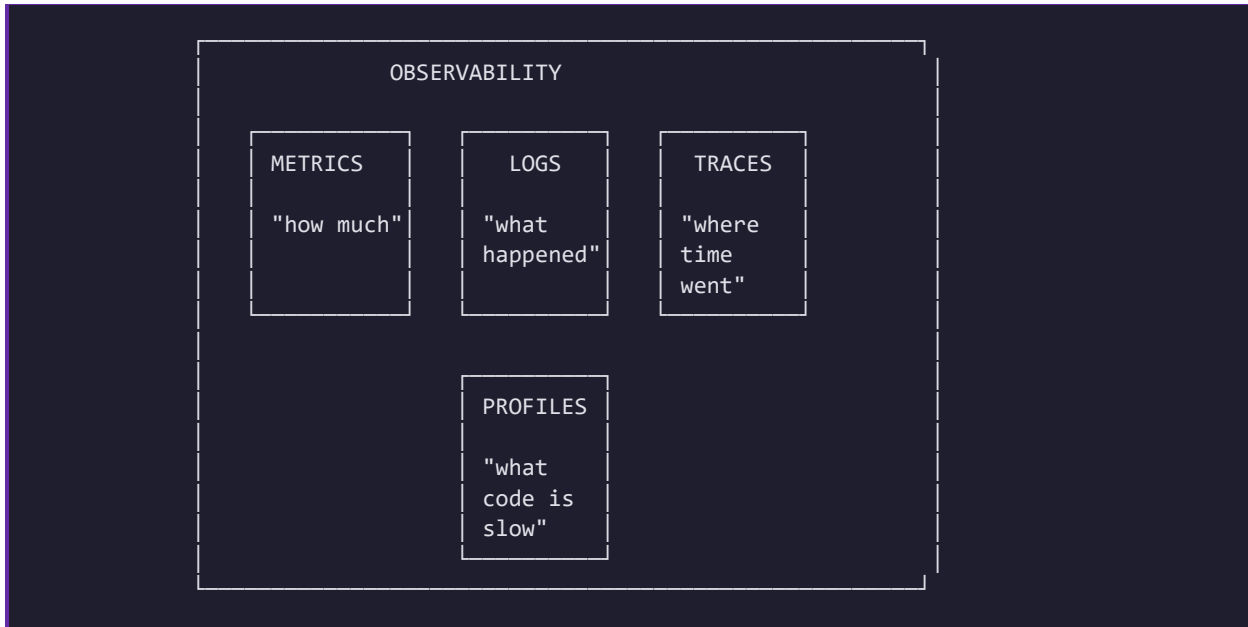
1.2 A Concrete Illustration

Two teams, one incident

Team A runs five dashboards: error rate, latency, throughput, CPU, memory. An incident happens. The dashboards all look green. The team is stuck — they have no way to ask "what changed?" beyond the five things they already track. They reach for SSH and grep, and the investigation takes hours. Team B runs the same five dashboards but also captures traces for every request, structured logs with rich tags, and continuous profiles. When their incident hits, they pivot from dashboards to traces, find one slow customer-specific endpoint, filter logs by that customer's account_id, and identify a corrupt cache entry — in twelve minutes. The difference is not that Team B had "better monitoring." It's that they had data they could ask new questions of.

1.3 The Three Pillars (Plus One)

Observability is built on three classical pillars — metrics, logs, traces — with profiles increasingly recognized as a fourth. Each captures a different facet of system behavior. None of them alone is enough; together they form a complete picture.



Metrics — the dashboards

Numeric measurements sampled at regular intervals, tagged with a small set of dimensions. CPU utilization, request rate, error count, queue depth, p99 latency. Cheap to store, fast to query, ideal for alerting on aggregate behavior. Limited by cardinality — every unique tag combination is a new time series, so high-cardinality fields like `user_id` cannot live on metrics.

Logs — the witness account

Discrete timestamped events, usually with much richer context than metrics — full error messages, stack traces, request bodies, user IDs. Each line stands alone, queried at search time. More expensive than metrics per data point, but the only way to recover detailed context after an incident.

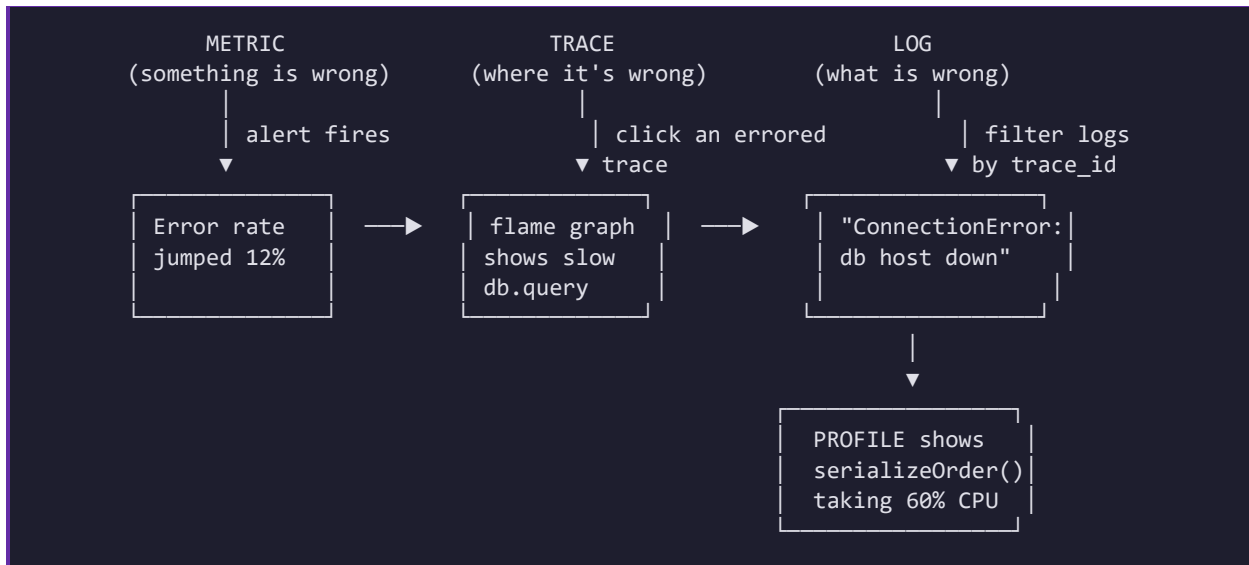
Traces — the journey

Records of a single request as it passes through every service, function, or database call. A trace is a tree of timed spans connected by a shared trace ID. Indispensable for distributed systems because no single service has the full picture. Module 5 covers traces in depth.

Profiles — the X-ray

Continuous, low-overhead recordings of CPU, memory, and lock contention inside a running process — function by function. Where a trace tells you "the payment-service span took 280ms," a profile tells you "180ms of those 280ms were spent in `serializeOrder()` at line 47." Profiles are what you reach for when self-time dominates a trace.

1.4 How They Complement Each Other



1.5 Which One When?

Question	Pillar to reach for first
How is the system behaving right now?	Metrics
Why did this one user's request fail?	Logs (filter by user)
Why is this endpoint slow?	Traces (flame graph)
Why is this service burning 80% CPU?	Profiles
Did the deploy regress performance?	Metrics (release-correlated)
Which downstream is failing?	Traces (service map)
What were the last 10 errors?	Logs
Is this within our error budget?	Metrics + SLOs

Practitioner truth

The mature observability practice doesn't choose between metrics, logs, traces, and profiles — it uses all four, with clear roles. Metrics drive alerting and dashboards. Traces drive cross-service debugging. Logs drive deep-context investigation. Profiles drive code-level optimization. Each is a different camera angle on the same system; you need them all to understand what you're looking at.

2. SLIs, SLOs, and SLAs

If observability gives you the ability to ask any question about your system, SLOs give you the discipline to ask the right one: "is our service good enough for the people who use it?" Service Level Objectives are how mature teams turn fuzzy goals like "keep it reliable" into concrete, measurable, accountable targets. This chapter introduces the three-letter trio — SLI, SLO, SLA — and the powerful idea of error budgets that follows from them.

2.1 The Three Letters

Term	Stands for	What it is	Audience
SLI	Service Level Indicator	A metric measuring some aspect of service quality	Internal — engineers
SLO	Service Level Objective	A target for an SLI over a window	Internal — engineers + product
SLA	Service Level Agreement	A contractual promise to customers, with penalties	External — legal + customers

The relationship: SLAs are what you sign with customers. SLOs are what you commit to internally — typically a bit stricter than your SLA, so you have margin. SLIs are the actual measurements that tell you whether you're meeting the SLO. Most SLAs are simple uptime promises; most useful SLOs measure something richer.

2.2 SLIs — Picking the Right Metric

A good SLI captures user-perceived quality, not internal engineering convenience. The four most common SLI categories:

Category	Definition	Example
Availability	Fraction of requests that succeeded	<code>successful_requests / total_requests >= 99.9%</code>
Latency	Fraction of requests fast enough	<code>requests_under_300ms / total_requests >= 95%</code>
Quality / Correctness	Fraction of responses that were correct	<code>responses_returning_valid_recommendations / total >= 99%</code>
Freshness	Fraction of data fresh enough	<code>queries_returning_data_under_5min_old / total >= 99%</code>

2.3 The SLO Formula

An SLO is composed of three parts. Write each one explicitly:

An SLI (the measurement) e.g. ratio of successful requests
A target (the goal) e.g. $\geq 99.9\%$
A window (the time horizon) e.g. rolling 30 days

Reads as: "99.9% of requests must succeed over rolling 30 days."

2.4 The Magic of 9s — And the Budget They Create

Reliability is often expressed as "three nines" (99.9%), "four nines" (99.99%), etc. Each nine sounds incremental but represents an order-of-magnitude jump in cost and complexity:

SLO	Allowed downtime / month	Allowed downtime / day
99% (two nines)	7h 18m	14m 24s
99.5%	3h 39m	7m 12s
99.9% (three nines)	43m 49s	1m 26s
99.95%	21m 54s	43s
99.99% (four nines)	4m 22s	8.6s
99.999% (five nines)	26.3s	0.86s

The right number is rarely "as many nines as possible." Two reasons: first, every extra nine costs roughly 10× more in engineering effort, redundancy, and operational complexity. Second, your customers can't perceive reliability that exceeds the rest of their stack — five-nine APIs are useless if their browser, their ISP, and their device average four nines.

2.5 Error Budgets — The Operational Genius

The most powerful idea in the SLO playbook is the error budget. If your SLO is 99.9%, your error budget is the remaining 0.1% — a finite, refilling quota of "badness" you're allowed in each window. Once that idea exists, every operational decision gains a quantitative frame:

- "Can we deploy a risky change?" → If we've used 30% of this month's error budget, yes. If 95%, no — freeze deploys.
- "Should we invest in more reliability work?" → If we routinely exhaust the budget, yes. If we routinely have 90% left, redirect to feature work.
- "Should we accept this dependency risk?" → Quantify worst-case budget consumption; decide.

```
SLO target:                99.9%
Error budget (1 month):    43m 49s of downtime allowed
```

```
If you spent 30m so far:
- Budget remaining:        13m 49s (32% left)
- Deploys: proceed normally
- Aggressive experiments: caution
```

```
If you spent 42m so far:
- Budget remaining:        1m 49s (4% left)
- Deploys: only critical fixes
- Aggressive experiments: postpone
```

2.6 SLA — The External Contract

An SLA is a legal commitment, usually with financial penalties: "if availability falls below 99.9% in a month, we credit 10% of monthly fees." Three rules of thumb:

1. Set the SLA looser than the SLO. If your internal SLO is 99.9%, your SLA might be 99.5% — giving you margin so internal misses don't become external refunds.
2. Measure the SLA exactly as you measure the SLO. If the SLA says "availability" and you measure availability differently, you'll have a customer dispute eventually.
3. SLAs are about edges, not averages. A single bad day can blow your monthly SLA. Plan for the worst hour, not the typical one.

2.7 SLOs in Datadog

Datadog's SLO product lets you define each of these and tracks them automatically:

- Pick an SLI from a metric or a monitor (e.g. service:checkout error rate < 1%).
- Set the target and window.
- Datadog shows real-time budget remaining, burn rate, history, and forecasts when the budget will exhaust.
- Configure alerts on burn rate ("alert if budget burning fast enough to exhaust before window ends") rather than just on the underlying metric.

2.8 Real-World SLO Story

From firefighting to budget-driven

A team operates a payments API. They have no SLOs. Every quarter, leadership asks "is the API reliable?" and the answer is a confident-sounding "yes" backed by no number. They begin defining SLOs: 99.9% availability, p95 latency < 400ms, both over rolling 30 days. Three months later they have data. Availability runs at 99.93% (slight margin); latency runs at 96% of requests under 400ms (slight margin). They share these numbers with leadership and the customer-success team for the first time. A month later a deploy regresses latency. Budget burn rate doubles overnight; alert fires;

the team rolls back within an hour and protects the budget. The same regression in the no-SLO world would have been noticed by customers two weeks later. The numerical framing changes the conversation from feelings to math.

3. Golden Signals

The four Golden Signals — Latency, Traffic, Errors, Saturation — come from Google's Site Reliability Engineering book. They are the smallest set of metrics that, together, give you a complete picture of a user-facing service. If you can only have four dashboards per service, these are the four. Datadog APM gives them to you automatically for every traced service; for non-traced services, you build them yourself.

3.1 The Four Signals

Signal	What it measures	Why it matters
Latency	Time to serve a request, as percentiles	User-perceived speed
Traffic	Demand on the service (RPS, throughput)	Demand level, capacity planning
Errors	Rate of failed requests	Quality of service
Saturation	How full the service is	Capacity headroom; predicts future failures

3.2 Latency — Percentiles, Not Averages

Latency is the time from request received to response sent. Always reported as percentiles, never as averages, because averages hide the long tail of slow users. A service with 100ms average latency might have p99 of 8 seconds — 1% of your users wait 8 seconds. That 1% is loud.

Percentile	What it means	Who feels it
p50 (median)	Half of users get faster, half slower	Typical user experience
p95	5% of users wait longer than this	Your power users notice it
p99	1% of users wait longer	Your largest customers feel it
p99.9	0.1% of users wait longer	At a million requests/day, 1,000 users

A surprising lesson from years of operating distributed systems: optimizing the median rarely matters; optimizing the tail almost always does. The slow tail is where customer complaints, support escalations, and lost sales live.

3.3 Latency — Success vs Failure

Always split latency into successful and failed requests, then report only the successful one. Otherwise a service that fails fast looks artificially "fast":

```
# Bad: one chart of all-request latency
service drops to 5ms latency      <- looks good!
... but error rate is 100%       <- everything is failing fast
```

```
# Good: split charts
successful_latency p95 = 320ms
failed_latency p95    = 8ms      (because timeouts succeed)
error_rate            = 100%     (so you know to trust failed_latency)
```

3.4 Traffic

Traffic measures demand — how much work the service is asked to do. The units depend on the service:

- HTTP service: requests per second.
- Database: queries per second.
- Message queue: messages produced/consumed per second.
- Streaming service: connections, bytes/second.
- Batch system: jobs started per minute.

Traffic by itself is rarely actionable, but it's the denominator for many other things. Error rate is errors/traffic. Saturation can be compared to traffic to predict when capacity will be reached. And anomalies in traffic itself are often the first sign of an upstream problem (your service didn't break — your customers stopped calling).

3.5 Errors

Errors are requests that failed to fulfill their contract. The shape of the definition matters:

- HTTP service: 5xx responses are errors. 4xx is configurable — 401s during normal browsing are not service errors; 401s caused by your bug are.
- Database: query exceptions, connection failures.
- Queue: messages on the dead-letter queue.
- Asynchronous: silent failures (no error response was generated). These are the hardest — instrument them deliberately.

Error rate is usually reported as a percentage of total traffic: errors / total. A 1% error rate is benign on a chat app; catastrophic on a payments API.

3.6 Saturation

Saturation is how full the service is. Subtler than the others, because every system has multiple resources that can fill up. Examples:

Resource	How saturation is measured	Threshold to watch
CPU	Utilization %	70% sustained = scaling pressure
Memory	Used / total	85% with no headroom = OOM imminent
Disk	Used / total + IOPS used	80% = plan for expansion

Resource	How saturation is measured	Threshold to watch
Connection pool	Active / max	80% = pool exhaustion soon
Thread pool / worker count	Busy / max	80% sustained = request queueing
Queue depth	Items waiting	Anything > 0 sustained = consumer is too slow
File descriptors	Open / max (ulimit)	80% = silent failures coming

The right saturation metric depends on what your service's bottleneck is. A CPU-bound API service watches CPU; a DB-bound service watches connections; a queue-driven worker watches queue depth. Many incidents are at heart saturation problems that nobody noticed until it failed.

3.7 The Golden Signals in Action — One Service Dashboard

Service: checkout (production)		
LATENCY p50: 120ms p95: 320ms p99: 580ms p99.9: 1.2s		TRAFFIC 450 req/s (steady)
ERRORS Rate: 0.4% Total: 1,800/day Top error: timeout to payment-svc		SATURATION CPU: 42% Mem: 61% DB pool: 18/50 (36%) Worker threads: 24/64 (37%)
--- All four green; no alerting condition. ---		

3.8 RED and USE — Two Subsets You'll Hear About

Two well-known frameworks are derivatives of the Golden Signals, popular in specific communities:

Framework	Signals	Best for
RED method (Tom Wilkie)	Rate, Errors, Duration	Request-driven services (APIs, web)
USE method (Brendan Gregg)	Utilization, Saturation, Errors	Resource-centric (hosts, databases)
Golden Signals (Google SRE)	Latency, Traffic, Errors, Saturation	General-purpose

They overlap. Don't get religious about which acronym to use; the substance is the same. The Golden Signals are the most general and the most widely cited.

3.9 Real-World Lesson

The four-signal triage

An on-call engineer is paged at 02:17. The alert says "checkout error rate above 1%." Before opening anything else, they pull up the four Golden Signals for the checkout service: errors are spiking to 4.5%, latency is normal (p99 = 600ms, no rise), traffic is normal, saturation is normal across all resources. Conclusion in 15 seconds: the service is healthy in every way except outcomes — so the failures aren't a capacity or a code-path issue. They're downstream. Engineer pivots to the service map, finds the payment-gateway dependency turning red, escalates to that team. Diagnosis began 15 seconds after the page. Without the Golden Signals reflex, the same engineer might have spent 20 minutes hunting through dashboards for which thing was wrong.

4. Datadog Building Blocks

Before going deeper into any single Datadog product, it pays to understand the structural pieces that show up everywhere: organizations, users, roles, API keys, integrations, agents, and tags. These are the platform's nouns. Every dashboard, every alert, every trace search refers to them. Getting these right at the start saves vast amounts of cleanup later.

4.1 Organizations and Sites

Your Datadog account lives in an Organization. The Organization is the top-level boundary: it owns billing, user roster, integrations, and all telemetry data. Most companies have one organization; large enterprises have several (one per business unit, one per major region) and use Datadog's Multi-Org Account to administer them together.

Each organization lives at a Site — a regional Datadog deployment with its own URL and data residency:

Site	URL	Data residency
US1 (default)	app.datadoghq.com	United States
US3	us3.datadoghq.com	United States (Azure)
US5	us5.datadoghq.com	United States
EU	app.datadoghq.eu	Frankfurt, Germany
AP1	ap1.datadoghq.com	Japan
US Gov	app.ddog-gov.com	US (FedRAMP)

Site selection is a one-way decision — you can't migrate organizations across sites without re-ingesting. Pick based on where your data is allowed to live, not where your customers are.

4.2 Users, Roles, and RBAC

Datadog's access model is role-based. A user can have one or more roles, and each role carries a bundle of permissions: which products the user can read, which they can modify, which integrations they can configure, and which sensitive data scopes they can see.

Default role	Permissions
Read Only	View dashboards, metrics, logs, traces — cannot edit
Standard	Read + create/edit dashboards, monitors, integrations
Admin	Standard + user management, billing, security
Custom roles	Compose any subset of fine-grained permissions

Practical RBAC patterns

- Engineers: Standard role + restriction queries that scope them to their team's services.
- Auditors: Read-only on specific indexes (audit log only).
- On-call rotations: Standard + write permission on monitors.
- Service accounts (automation): Custom role with only the API surface they need.
- Executives / dashboards-only viewers: Read-only on a curated dashboard list.

4.3 API Keys and Application Keys

Two kinds of keys exist, and confusing them is a security mistake worth avoiding:

Key type	Used by	Scope
API Key	Agents, intake endpoints, ingestion clients	Allows sending data IN
Application Key	Scripts, CI/CD, automation, terraform	Allows pulling data OUT and managing config

API keys come in two flavors: full-power keys, and scoped keys (logs-write only, metrics-write only). Use scoped keys wherever possible — particularly for any key that ships in an artifact you don't fully control (mobile SDK, edge function, container image).

Security hygiene

Rotate API and Application keys quarterly. Audit them annually — every key has a creator and a last-used date. Keys that haven't been used in 90 days should be revoked. The Datadog audit trail (Module 4, Section 8) shows every key creation, every key use; integrate this with your SIEM.

4.4 Integrations

An integration is a pre-built connection to a technology — AWS, GCP, Kubernetes, Postgres, Redis, GitHub, Slack, PagerDuty, etc. Datadog ships 750+ of them. Each integration:

- Knows how to collect the right metrics from the target (specific endpoints, APIs, MBeans).
- Ships pre-built dashboards in the UI.
- Ships pre-tuned monitor templates.
- Adds the right tags to its telemetry automatically.

There are two flavors:

Type	How it works	Examples
Agent-based	Datadog Agent runs a check that polls the target	Postgres, Redis, Nginx, Kafka

Type	How it works	Examples
Cloud / API-based	Datadog polls the cloud provider's API	AWS, GCP, Azure, GitHub, Slack

4.5 The Datadog Agent

The Agent is the workhorse — a small Go daemon that runs on every host, container, or Kubernetes node. It collects metrics, logs, traces, profiles, processes, and network data and securely forwards them to the Datadog backend.

The Agent is composed of several sub-processes (Module 2 covers them in depth):

Sub-process	Role
Core Agent	System and integration metrics
Trace Agent	APM trace ingestion on :8126
Process Agent	Live process / container inventory
Logs Agent (in core)	File tailing, container stdout, sockets
System Probe	eBPF kernel telemetry (NPM, USM)
Security Agent	Cloud Workload Security

4.6 Tags

Tags are the dimensions Datadog uses to slice every piece of data. Every metric, log, trace, process, and host carries a set of key-value tags. They are the most-leveraged feature of the platform and the one most-often abused. Modules 2, 3, and 4 each have a chapter on tags from their respective angles; here is the platform-wide view.

Tag	Source	Used for
env	Set on every agent and app	Filter by environment
service	Set on app, propagates to traces/logs	Service-scoped views
version	Set on app at deploy time	Deployment correlation
host	Auto-set by agent	Per-machine queries
region, az, datacenter	Auto-set by cloud integration	Geographic / failure-domain
team, cost_center, app	Set manually for ownership	Routing alerts, FinOps
pod_name, container_id, k8s_namespace	Auto-set by Kubernetes integration	Per-pod queries

4.7 Unified Service Tagging

Of all the tags, three are sacred: env, service, version. Datadog calls this set Unified Service Tagging because when all three are set consistently across metrics, logs, and traces, they form a join key that links the pillars together. With Unified Service Tagging:

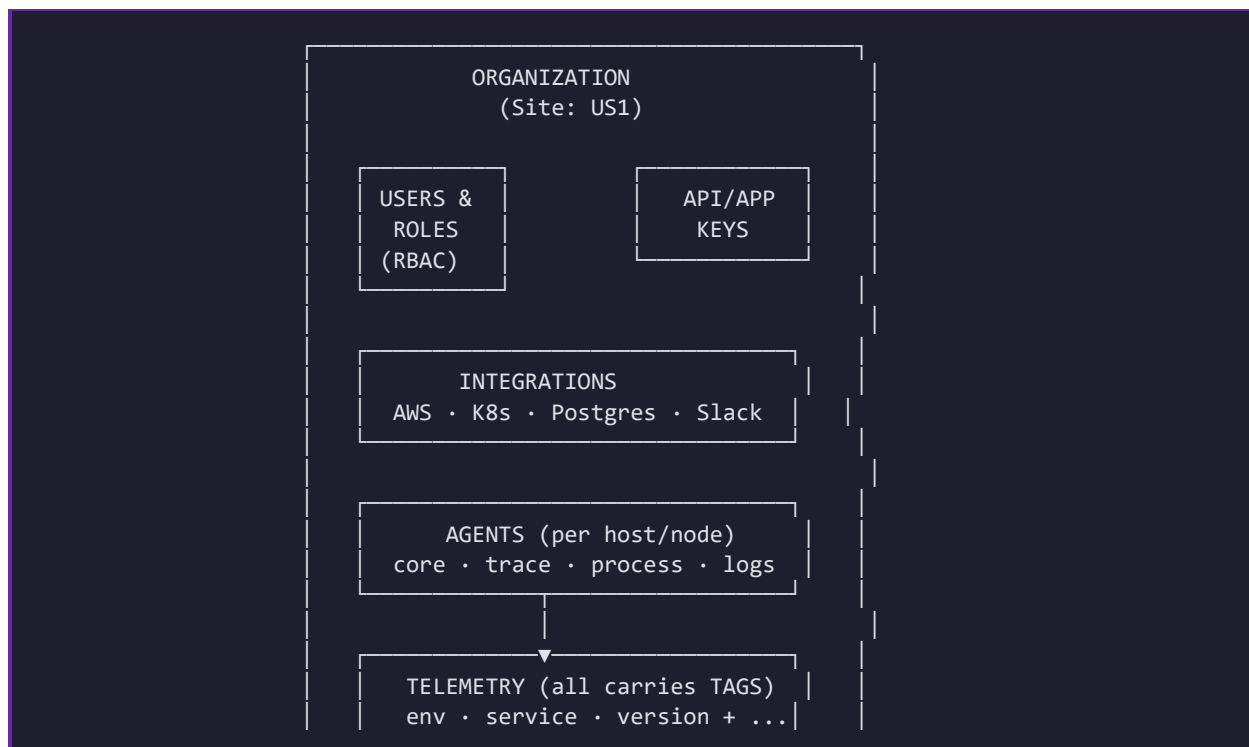
- Click a slow trace → see the logs from the same env+service+version automatically.
- Click a metric anomaly → jump to traces in the affected version.
- Filter a dashboard by version to see how a new deploy compares to the previous one.
- Release markers correlate every chart with deploys.

Setting all three correctly is the single highest-leverage thing a platform team can do. Without it, you have three siloed data stores that happen to be in the same UI. With it, you have a unified observability platform.

How to set them

Mechanism	Example
Environment variables on the app	DD_ENV=prod, DD_SERVICE=checkout, DD_VERSION=2.1.4
Pod labels (Kubernetes)	tags.datadoghq.com/env: prod, ...service, ...version
Agent-level for all telemetry on the host	tags: env:prod (in datadog.yaml)
OTel resource attributes	deployment.environment, service.name, service.version

4.8 Putting It Together





5. Data Flow and Cost Levers

Datadog's pricing model has earned a reputation for being opaque. It isn't — but it does have many surfaces. Each telemetry type (metrics, logs, traces, RUM, synthetics, etc.) has its own pricing dimension, and each one has different cost levers. This chapter is the platform-wide map of how data flows through the system, where each step is billed, and the handful of decisions that drive 80% of every team's spend.

5.1 The Universal Pipeline



5.2 The Per-Product Cost Picture

Product	Primary cost driver	Secondary cost driver
Infrastructure	Per host / per million container minutes	Custom metrics (separate SKU)
Custom Metrics	Per unique time series	(Cardinality is everything here)
APM	Per indexed span (sampled)	Per host with APM enabled
Logs	Per GB ingested + per million indexed events	Retention duration multiplier
Continuous Profiler	Per profiled host hour	—
RUM	Per session	Per replay session (if enabled)
Synthetics	Per test run	Browser vs API multiplier
Network Performance	Per host with NPM enabled	—
DBM (Database Monitoring)	Per database host	—

5.3 The Five Most Important Cost Levers

Lever 1 — Metric cardinality

This is the single largest source of unexpected bills. Every unique combination of tag values on a metric is a separate time series, and each one is billed. Tags like `env` (5 values) and `service` (200 values) yield 1,000 series — fine. Adding `user_id` (a million users) makes it a million series per metric. Always audit:

- Which custom metrics exist and how many series they produce.
- Which tags are on each — flag any unbounded-cardinality tag.
- Whether a high-cardinality metric is actually queried anywhere; if not, drop it.

Lever 2 — Log indexing vs ingestion

You pay separately for ingesting a log (per GB) and for indexing it (per million events). Most teams overpay 5-10× on indexing because they index logs nobody ever reads. The fixes are exclusion filters and sampling — covered exhaustively in Module 4.

Lever 3 — APM trace sampling

Traces are billed per indexed trace. At 100% sampling, a service with 1,000 RPS generates 1,000 traces/second to index. With smart sampling — keep 100% of errors and slow traces, 1-10% of fast successful ones — you cut volume 10-90× without losing signal. Module 5, Section 8 has the details.

Lever 4 — Retention duration

Logs and indexed traces are billed both for events and for retention duration. A 30-day retention costs roughly 2x a 15-day retention for the same volume. Split into multiple indexes (Module 4): short retention for routine logs, long retention for audit/compliance.

Lever 5 — Host vs container counts

Infrastructure billing is per host. Whether to enable per-container billing depends on density. A single host running 50 containers might be cheaper on per-host billing than per-container; the opposite is true for sparsely-packed clusters. Watch your bills monthly and switch SKUs when the break-even shifts.

5.4 Cost Allocation

In organizations with many teams, the hardest cost question is not "how much do we spend?" but "who is spending what?" Tag-based attribution is the answer:

```
# Tag every service with cost-center and team
env:prod service:checkout team:payments cost_center:1234

# Then in the Datadog Usage page (Admin):
- See custom-metric count by service tag
- See log volume by team tag
- See APM trace volume by service tag

# Export to your FinOps tool / data warehouse for charge-back
```

5.5 A Real Cost-Optimization Story

70% reduction in three weeks

A SaaS company hit \$X00k / month on Datadog. Audit found: (1) custom metrics had a tag with 200,000 unique values added accidentally by a developer — single metric was 200k series; (2) all log indexes were at 30 day retention by default, including dev environments; (3) APM was at 100% sampling for every service including a marketing-page tracker that emitted 10,000 RPS of useless successful pageviews. Fixes: removed the bad tag (1 hour, ticket); split indexes by env and lowered dev to 3-day (1 day, IaC change); set APM sampling rules per service (1 week, with each team reviewing). Result: bill dropped 70%, no observable user-facing capability lost.

5.6 Cost Discipline as a Practice

- Review the Datadog Usage page monthly — it shows every SKU broken down.
- Set Custom Metric quotas per service to prevent runaway tags.
- Configure spend alerts at the org level — 80% and 100% of budget.
- Quarterly cleanup: delete unused dashboards, unused monitors (their queries cost money to run), unused integrations.
- Bake cost into code review: "this PR adds a new metric — what's its cardinality?"

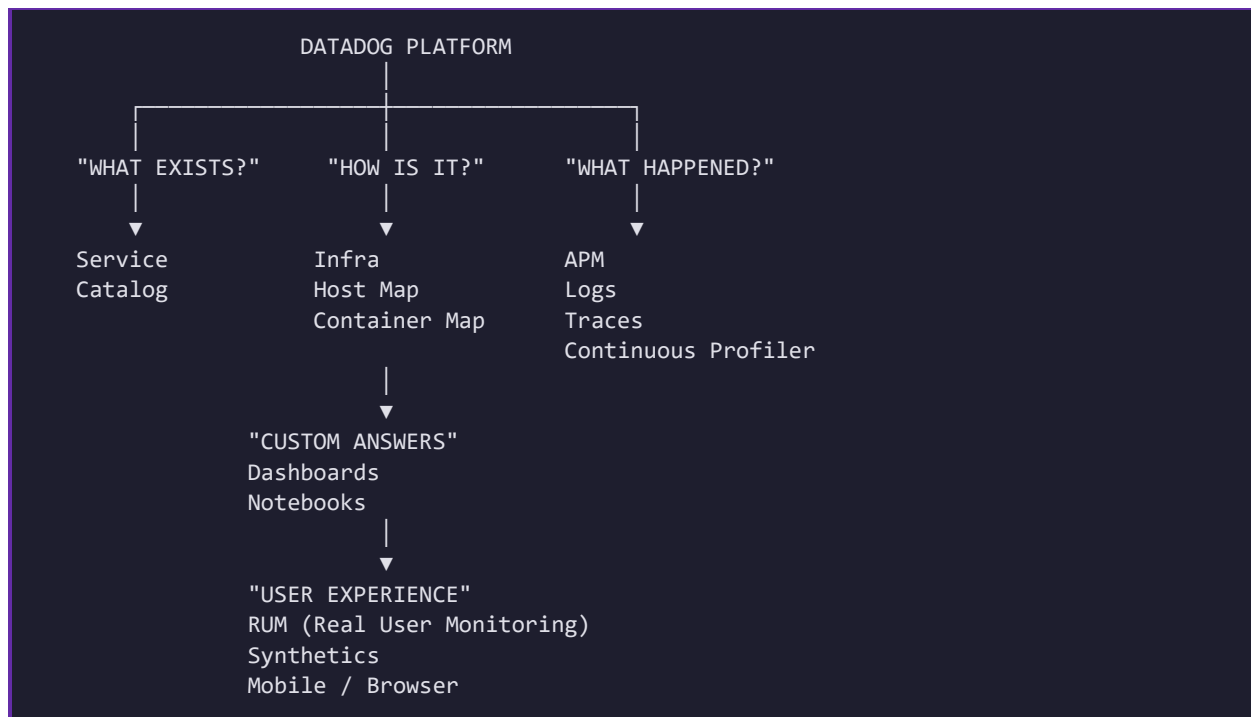
Cultural truth

Observability cost discipline is most effective when it's distributed, not centralized. A central FinOps team can audit once a quarter and find the worst offenders; only the service teams themselves can prevent the problem from coming back. Tools (quotas, dashboards, audit queries) work, but a culture of "think about cardinality" is what sustains them.

6. Key Platform Views

Datadog's product surface is wide. The same telemetry is presented through a dozen different views, each optimized for a different question. A new user can get lost in the menu; an experienced user moves between views fluidly during an incident. This chapter is the tour — what each major view is for and when to reach for it.

6.1 The View Map



6.2 Service Catalog

The Service Catalog is the inventory: every service you operate, with metadata about ownership, tier, dependencies, runbooks, repository links, SLOs, and recent deploy history. It's the answer to "what services exist in our org and who owns them?"

- Populated automatically when services emit traces (auto-discovered).
- Enriched manually or via a `service.datadog.yaml` file checked into the service's repo.
- Cross-linked to APM (click service → its APM page), to monitors, to dashboards, to SLOs.
- Becomes the single source of truth for "who owns this?" during incidents.

6.3 Infrastructure (Host & Container Map)

The Infrastructure section visualizes your fleet. Two main views:

View	Shape	Best for
Host Map	Hexagonal grid; one hexagon per host	VM / bare-metal fleets; spot the unhealthy host
Container Map	Hexagonal grid; one per container	Container fleets; find the noisy pod
Process List	Per-process tree from process-agent	Find the JVM eating CPU
Network Performance	Service-to-service network flows (eBPF)	Connection-level issues

The map view is the right starting point during infrastructure-level incidents. A node turns red — you know exactly which one, can click in, see its processes, its containers, its disk and network status.

6.4 APM

The APM section is the home for traces. Covered in full in Module 5; for orientation, the main sub-views:

- Service Map — runtime topology of all your traced services.
- Services list — every service with its golden-signals dashboard.
- Trace Search — find specific traces with filters.
- Service-level dashboards — auto-generated per service.
- Continuous Profiler — function-level CPU and memory profiles.
- Error Tracking — grouped issues from errored spans.
- Database Monitoring (DBM) — query-level performance for your DBs.

6.5 Logs

Covered fully in Module 4. The two main sub-views:

- Log Explorer — filter, group, and search indexed logs.
- Live Tail — real-time stream including excluded logs.

Plus configuration views: Pipelines (Section 5 of Module 4), Indexes (Section 6), Generated Metrics, Archives.

6.6 Real User Monitoring (RUM)

RUM captures telemetry from the actual user's browser or mobile device: page load timings, JS errors, user interactions, sessions, and (optionally) a video-like replay of the user's actions. While APM tells you how the backend behaved, RUM tells you how the user experienced it. The two together close the loop:

- APM: "the API responded in 200ms."
- RUM: "but the user saw a blank page for 4 seconds because of slow JavaScript on the client."

Modern RUM also correlates with APM via shared trace IDs — click a slow page load in RUM, see the backend trace for the AJAX requests that triggered it.

6.7 Synthetics

Synthetic tests simulate users from Datadog's global locations on a schedule. Two types:

Type	What it does	When to use
API tests	HTTP/gRPC requests with assertions	Endpoint uptime, basic functionality
Browser tests	Scripted real Chromium sessions	Multi-step user journeys (login, checkout)

Synthetics fill the gap RUM and APM can't: monitoring when no real users are around. A scheduled login-checkout test running every 5 minutes from three regions tells you about uptime continuously, even at 3 AM on a Sunday.

6.8 Dashboards

Dashboards are the customizable surface — composed of widgets (charts, tables, query-value tiles, log streams, etc.) you arrange to tell a specific story:

- Per-service dashboards — owned by the service team, reviewed weekly.
- Per-product dashboards — "how is checkout doing" — composed across services.
- Executive dashboards — SLO compliance, error budgets, business KPIs.
- Incident-response dashboards — pre-built views for known failure modes.

Dashboards in Datadog support template variables (filter the whole dashboard by env, by region, by service) and time-shifting (compare this hour to the same hour last week). The combination makes them genuinely powerful, not just chart galleries.

6.9 Notebooks

Notebooks are interactive documents that mix text, charts, queries, and runbooks. They're typically used for:

- Incident post-mortems — the live timeline of an incident as it unfolded, with charts.
- Investigations — ad-hoc "let's look at this together" sessions you save and share.
- Runbooks — "when X alert fires, run these queries in this order."
- Capacity-planning analyses — quarterly reviews with prose and supporting evidence.

Dashboards are for ongoing operations; notebooks are for one-shot or periodic narratives. The boundary blurs — but rule of thumb: if you'd put text into it, it's a notebook.

6.10 The Incident-Response Tour

A page fires for "checkout error rate above 1%."

The on-call's path through the UI:

1. CLICK ALERT -> opens the monitor page
 - sees recent history, recent deploys
2. SERVICE CATALOG -> "who owns checkout?"
 - finds team, on-call rotation, runbook
3. APM SERVICE PAGE -> golden signals for checkout
 - confirms errors up, latency normal
4. APM SERVICE MAP -> "is this propagating?"
 - finds downstream payment-svc is also red
5. APM TRACES -> filter errors, open flame graph
 - identifies which downstream call fails
6. LOGS (correlated) -> "what does the error message say?"
 - reads stack trace, finds the cause
7. NOTEBOOK -> log the timeline as you go
 - becomes post-mortem material later

Six tabs, three minutes

Mature on-callers do this without thinking — Service Catalog, APM service, Service Map, Traces, Logs, Notebook open in adjacent tabs. The platform is designed for this flow. Practicing it on game days (not just real incidents) is how teams turn good tools into fast outcomes.

7. Unified Telemetry Model

Datadog markets itself as a "unified observability platform." The word that matters there is unified. A unified platform is not just one that happens to host metrics, logs, traces, and profiles in the same UI — it's one where those four data types share a vocabulary, a tag schema, and join keys, so a question that starts in one type can be answered in another. This chapter explains how that unification actually works under the hood, because understanding it changes how you instrument.

7.1 The Problem It Solves

Pre-unified observability looked like this: metrics in one tool (Prometheus or Graphite), logs in another (Splunk or ELK), traces in a third (Jaeger or Zipkin), profiles only in dev tooling. Each had its own concept of "service," its own tag rules, its own query language. Asking a cross-pillar question — "what does p99 latency on checkout look like at the same time the error logs spike?" — required three separate queries, manual time alignment, and an analyst's attention. The seams between tools were where insights died.

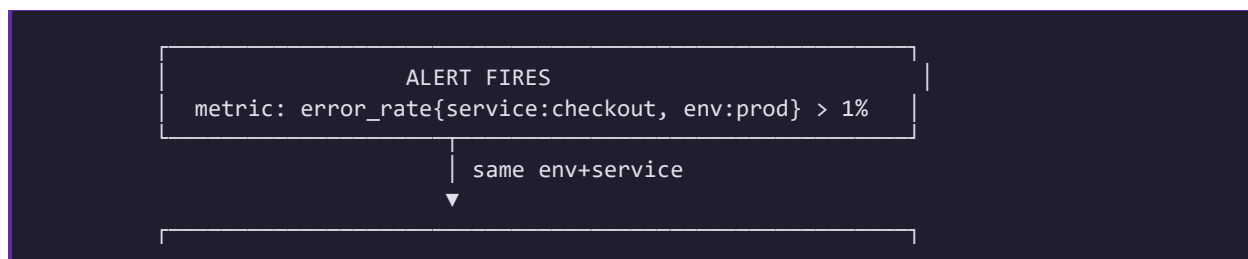
7.2 The Join Keys

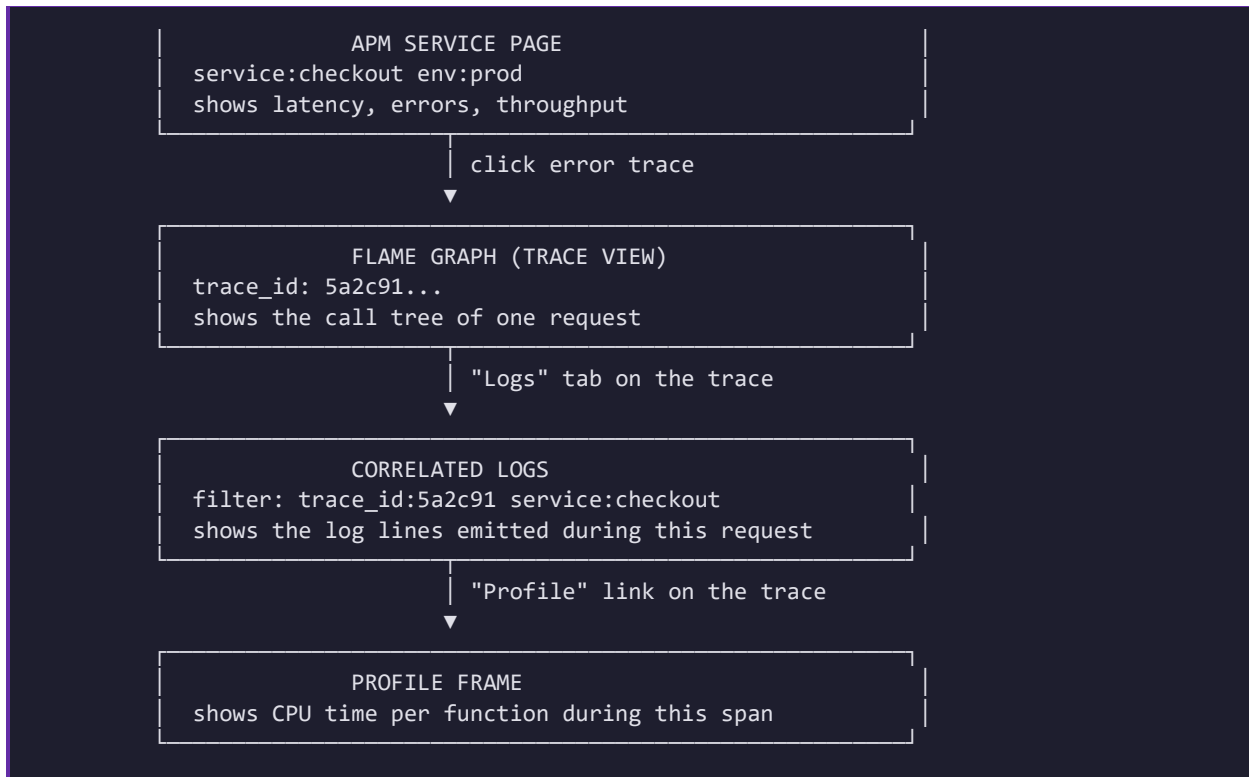
In a unified model, every piece of telemetry shares a small set of join keys. Datadog's are:

Join key	Carried on	Used to correlate
env	Metrics, logs, traces, profiles, RUM	Filter all pillars by environment
service	Metrics, logs, traces, profiles, RUM	Show all data for one service
version	Metrics, logs, traces, profiles, RUM	Compare data across deploys
host	Metrics, logs, traces, profiles	Find data emitted by one machine
trace_id	Logs, traces	Connect logs to the request they describe
span_id	Logs, traces	Connect logs to the specific operation
session_id	RUM, logs, traces	Follow one user session end-to-end

env, service, and version together are called Unified Service Tagging. Setting them consistently is the single highest-leverage instrumentation decision a team makes — covered in detail in earlier modules, but worth repeating here because of how much it enables.

7.3 The Cross-Pillar Joins in Action





That's five pillars (metric → APM → trace → log → profile) connected by shared tags, in five clicks. Every step is one join key away from the next. This is what unified means in practice.

7.4 Three Things to Get Right

1. Use the same env, service, version across the stack

Set them on the agent, on the app, in pod labels, in OTel resource attributes. They must match — env:prod in your logs but env:production in your metrics breaks the joins. Standardize on one form and enforce it in CI.

2. Inject trace IDs into log records

DD_LOGS_INJECTION=true (or equivalent for your language) makes the tracer add dd.trace_id and dd.span_id to every log line. Without these, the logs-from-trace view shows nothing. Module 4 Section 4 covers this in detail.

3. Use consistent attribute names

If your traces tag spans with @user.tier but your logs tag with @customer_tier, you can't filter both at once. Pick canonical names — typically following the OpenTelemetry semantic conventions — and remap in your pipelines if necessary.

7.5 The Failure Mode of Disunification

What goes wrong without unified tagging

A team that hasn't standardized has, after a year: 200 services, of which 80 use env, 60 use environment, 40 use ENV, and 20 set nothing. service is set on 150 of them, with various spellings

(checkout, Checkout, checkout-api, checkout_v2). The result: org-wide dashboards are impossible to build because no single filter captures the data. Every team has their own dashboards, each of which works only for their services. Incident response becomes a per-team exercise, never a shared platform view. Six months of cleanup work later, the team standardizes — and the platform suddenly becomes the unified system it was always supposed to be. The lesson: enforce standards early; you can't add them retroactively without significant pain.

7.6 Beyond Datadog — OpenTelemetry Semantics

OpenTelemetry's semantic conventions are the broader-industry version of this discipline: standardized attribute names for common concepts (`http.method`, `http.status_code`, `db.system`, `messaging.system`, etc.) so any tool can understand any other tool's data. Datadog aligns with OTel conventions and adds its own (the `@` prefix attribute namespace). For new instrumentation, prefer OTel semantic conventions — they're more portable and they're the direction the industry is moving.

8. Datadog vs Other Observability Tools

Datadog is the dominant unified observability platform in the SaaS-buying market, but it isn't the only one. Most enterprises evaluate it against Dynatrace, New Relic, Splunk, and increasingly the open-source/OTel stack (Grafana + Tempo + Loki + Prometheus). This chapter is the honest comparison — the strengths, the trade-offs, and the cases where each tool is the right pick.

8.1 The Major Players

Tool	Heritage	Strongest at
Datadog	SaaS-native, started with infrastructure monitoring	Unified breadth, integration ecosystem, fast UI
Dynatrace	Enterprise APM (origins in AppMon)	AI-driven root cause, deep Java/.NET, on-prem-friendly
New Relic	Pioneer of SaaS APM	Application performance, query-based UI (NRQL)
Splunk	Log analytics roots; broader via acquisitions	Logs at extreme scale, SIEM/security overlap
OSS Stack (Grafana et al.)	Open-source ecosystem	Vendor independence, customization, cost at small scale

8.2 Side-by-Side on Key Dimensions

Dimension	Datadog	Dynatrace	New Relic	Splunk	OSS Stack
Metrics	Strong	Strong	Strong	Via SignalFx (acquired)	Prometheus — strong
Logs	Strong	OK (added later)	Strong	Best-in-class scale	Loki — OK
APM / Tracing	Strong	Best-in-class auto-instrument	Strong	Via SignalFx — OK	Tempo — improving
RUM / Browser	Strong	Strong	Strong	Limited	Faro — early
Synthetics	Strong	OK	Strong	Limited	Blackbox exporter — basic
AI / anomaly	Watchdog, Bits AI	Davis (mature AI)	Lookout, AI	ITSI / Observability AI	Limited
Ease of getting started	Excellent	OK (enterprise sales)	Good	Steep learning curve	DIY

Dimension	Datadog	Dynatrace	New Relic	Splunk	OSS Stack
Integration count	~750	~600 (OneAgent auto)	~600	Varies by add-on	OTel-based, growing
UI / UX	Modern, fast	Modern but dense	Modern	Older feel	Grafana — strong
Pricing model	Many SKUs	Per-host (DEM)	Per-user + ingest	Per-GB ingested	Free + ops cost

8.3 Datadog vs Dynatrace

The most common head-to-head in enterprise procurement. Both are strong, but they sit on different ends of a spectrum:

Aspect	Datadog	Dynatrace
Setup model	Many small agents and integrations, you choose	OneAgent — one binary, auto-discovers everything
AI	Watchdog + Bits AI — pattern-matching + LLM	Davis — causal AI with deeper RCA
Customization	More query power, more knobs	More opinionated; less to tune
Platform breadth	Wider — security, real-user, synthetics, etc.	More focused on application performance
Best when	Building observability culture across a SaaS-native org	Enterprise with deep Java/.NET, wants RCA out-of-box
Pricing perception	Many SKUs, can be surprising	Per-host DEM model, predictable but expensive

8.4 Datadog vs New Relic

New Relic was the original SaaS APM. They've broadened over time but APM is still the heart. The most important contrast:

- New Relic's UI is built around NRQL — a SQL-like query language. Powerful for analysts; intimidating for new users.
- Datadog's UI is more click-driven, with query interfaces available but rarely needed.
- New Relic has pioneered a per-user + ingestion pricing model recently — friendly to teams with few users and lots of data; less friendly to large engineering orgs.
- Datadog's breadth across non-APM telemetry (RUM, Synthetics, Security, Network Performance, DBM) is wider.

8.5 Datadog vs Splunk

Splunk's roots are log analytics; their APM and metrics products came via the SignalFx acquisition. The split:

- If your primary need is logs at extreme scale — petabytes, complex SPL queries, SIEM use cases — Splunk is still best-in-class.
- If you need a unified platform across metrics, logs, traces, RUM, synthetics, security — Datadog's unification is tighter.
- Splunk's pricing is per-GB ingested, which makes their cost easier to predict for log-heavy workloads.
- Splunk's SIEM (Enterprise Security) is a stronger security product than Datadog's Cloud SIEM, though Datadog is closing the gap.

8.6 Datadog vs the Open-Source / OTel Stack

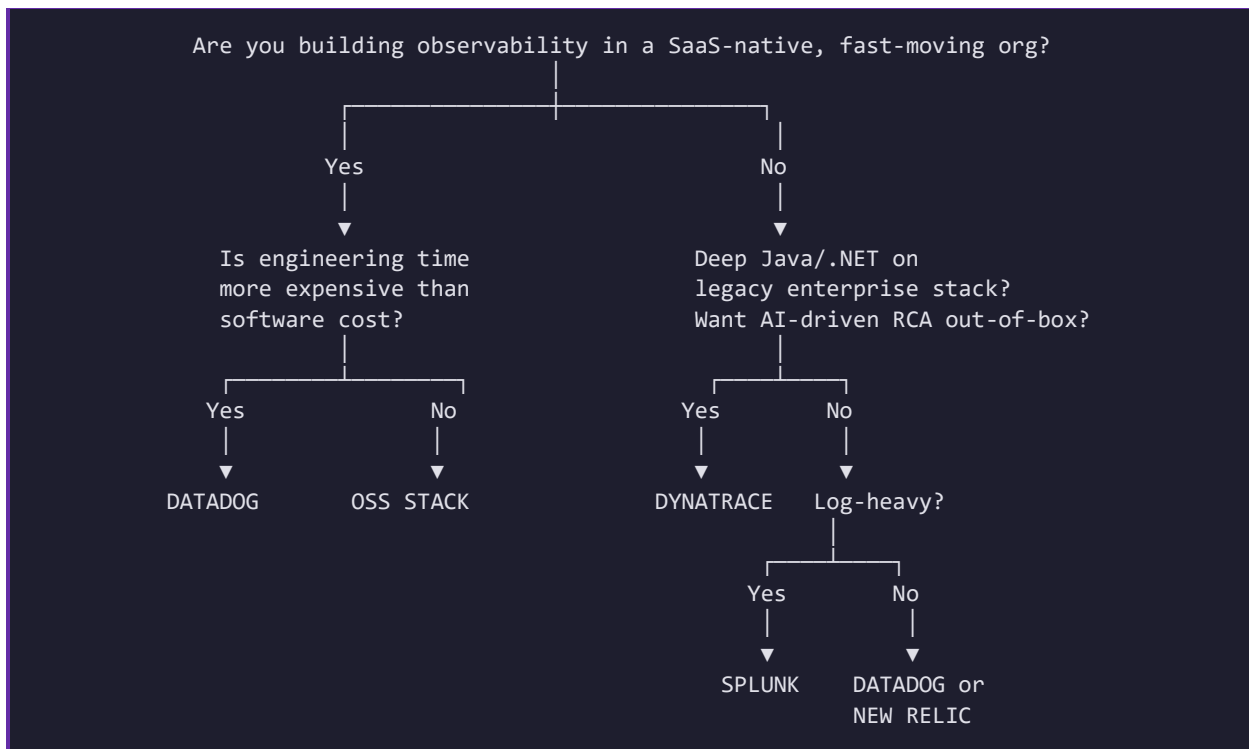
The most strategic comparison. The open-source stack — Prometheus for metrics, Loki for logs, Tempo for traces, Grafana for dashboards, all glued by OpenTelemetry — is genuinely viable in 2026 and increasingly chosen by cost-conscious or independence-conscious teams.

Aspect	Datadog	OSS Stack (Grafana + OTel)
Time to first chart	Minutes	Days to weeks to set up
Ongoing operational cost	Predictable subscription	Engineering hours to operate
At small scale (< 100 hosts)	Expensive overhead	Cheaper after setup
At large scale	Pricing curves up; volume discounts negotiable	Linear infrastructure cost (no licensing)
Vendor lock-in	Significant (proprietary SDKs unless OTel)	None — fully portable
AI / anomaly features	Mature	Limited or DIY
Integrations	750+, polished, supported	OTel-based, varying quality
Best for	Teams that want to focus on their product, not their observability stack	Teams with strong infra culture, cost-sensitive, sovereignty needs

The pragmatic position

There's a credible middle ground: use OpenTelemetry SDKs for instrumentation (vendor-portable) but send to Datadog as the backend. You get Datadog's mature UI, AI, and integration ecosystem with the ability to switch backends in the future without re-instrumenting every service. This is the most defensible position for most teams in 2026 — keep the keys to the kingdom (your instrumentation) under standard, portable control.

8.7 Choosing — The Decision Tree



9. AI Assistants in Datadog

AI in observability has gone from a marketing phrase to a daily-use feature in the last two years. Datadog ships three distinct AI capabilities — Watchdog, anomaly/forecast detection, and Bits AI — that serve different purposes. This chapter explains what each does, how to use it, and the realistic boundaries of each.

9.1 The Three Things, At a Glance

Feature	What it does	When it helps
Watchdog	Automatic anomaly detection across metrics, logs, traces, and deployments — no configuration	Catches issues you didn't have an alert for
Anomaly / Forecast	Configurable per-metric statistical detection	Specific metrics with strong seasonality or trend
Bits AI	LLM-based assistant; conversational queries, summarization, code generation	Querying data without learning the query language; incident triage

9.2 Watchdog

Watchdog is Datadog's always-on autonomous anomaly detector. You don't configure it; it watches every metric, error trace pattern, and log volume in your account and surfaces deviations. The output is a feed of Watchdog Stories — each one a flagged anomaly with a chart, the affected services, and a confidence score.

What Watchdog catches

- Metric anomalies: a service's latency, error rate, or throughput deviating from its learned baseline.
- Trace anomalies: a previously-rare error type suddenly common; a normally-fast resource turning slow.
- Log volume / error pattern anomalies: a service that suddenly logs 50× more errors of a specific type.
- Deployment regressions: post-deploy degradation that you didn't have an alert for.
- Infrastructure anomalies: unusual CPU, network, or disk patterns at the host or container level.

How to use Watchdog

Two patterns:

4. Reactive — when an incident strikes, open the Watchdog page filtered to your service. You often see the anomaly Watchdog already flagged, with context ("this started 23 minutes ago, affects 3 endpoints").
5. Proactive — review the Watchdog feed weekly; flagged stories that nobody escalated may be silent issues. Particularly useful for new services that don't have hand-built alerts yet.

Limitations

- Needs at least a week or two of stable history to build baselines.

- Noisy on rapidly-changing systems (a service in active development can produce false-positive Watchdog stories regularly).
- Not a replacement for tuned monitors — it's a safety net, not the primary alerting layer.

9.3 Anomaly and Forecast Detection

Beyond Watchdog's autonomous detection, Datadog provides explicit anomaly and forecast monitor types that you configure per metric. Use these when:

- The metric has a strong daily or weekly seasonality (e.g. login attempts) — a static threshold would alert every night.
- You want to detect drift earlier than a static threshold could (e.g. "alert when error rate deviates 3 standard deviations from rolling 7-day baseline").
- You want to forecast: "alert when projected disk usage will hit 100% in the next 48 hours."

```
# Anomaly monitor query (Datadog)
avg(last_5m):anomalies(
  sum:trace.http.request.errors{service:checkout},
  'basic', # algorithm: basic / agile / robust
  4        # std deviations of tolerance
) >= 1

# Forecast monitor query
avg(next_4h):forecast(
  sum:system.disk.used{role:db},
  'linear', 1
) >= 100
```

Anomaly and forecast monitors are powerful but require attention. Tune them on historical data first; deploy with notifications routed to a low-priority channel for a week; promote to PagerDuty only after they've proven their signal-to-noise ratio.

9.4 Bits AI

Bits AI is Datadog's LLM-based conversational assistant. You can ask it questions in natural language and it generates queries, summarizes incidents, suggests next investigation steps, and even generates code (Terraform, monitor config, query syntax). The release was a clear step beyond Watchdog's narrow-AI roots — Bits is a general-purpose copilot for the platform.

Things Bits AI is good at

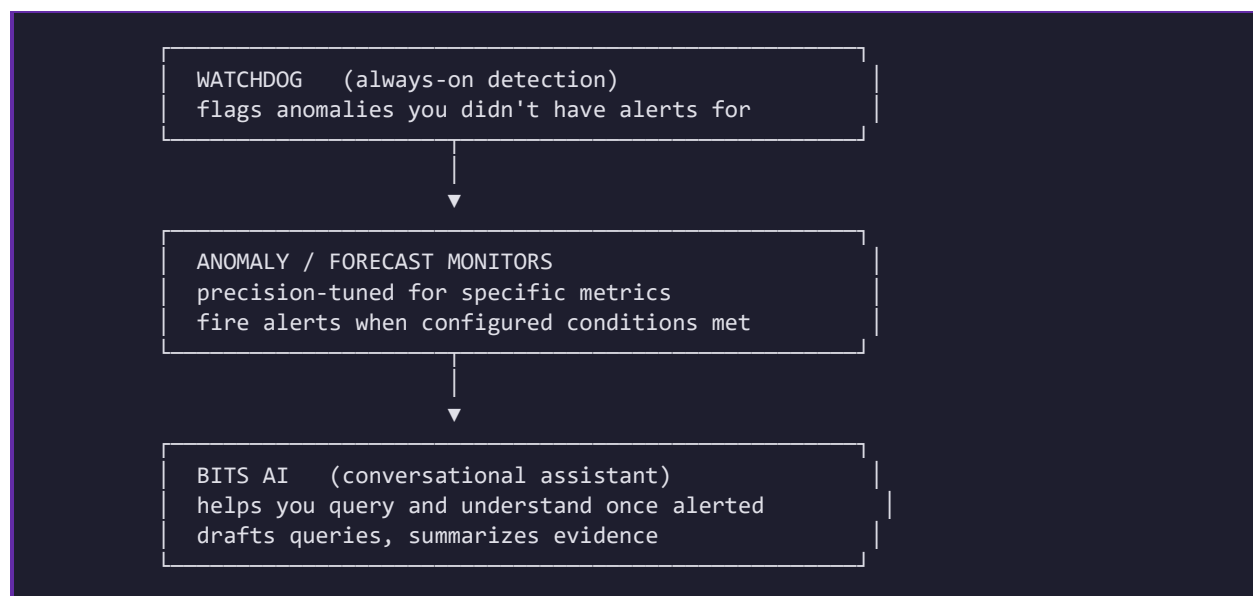
- "Show me error rate for checkout in the last hour grouped by version" — generates the query and the chart.
- "What does this trace tell me?" — summarizes a flame graph in plain English.
- "Why might this monitor be firing?" — looks at correlated metrics, logs, traces, and deploys.

- "Write a Datadog monitor that alerts on error rate above 1% with a 10-minute window" — drafts the monitor JSON/Terraform.
- Drafting incident-summary notebooks during or after an incident.

Things to remember

- It's an LLM. Verify the queries it generates — incorrect tag filters or missed aggregations are possible.
- Privacy: enterprise plans typically include zero-retention configurations for AI queries; verify yours.
- Bits AI doesn't replace platform knowledge — it accelerates the practitioner. A team that doesn't understand traces won't suddenly understand them through Bits.
- Like any LLM, it works best with specific, scoped questions. "Why is everything slow?" is too vague; "What's the p99 latency change on payment-service between version 2.1.3 and 2.1.4?" is actionable.

9.5 Three Layers Working Together



9.6 The Realistic Outlook

Where AI helps and where it doesn't

AI in observability is genuinely useful — anomaly detection finds problems faster than humans tuning thresholds, and LLM assistants make the platform accessible to people who don't have Datadog Query Language memorized. But it doesn't replace the fundamentals. A team with clean tags, well-structured logs, traces with proper context propagation, and tuned monitors gets enormous lift from Watchdog and Bits. A team without those foundations gets noisy AI on top of bad data. Invest in the data quality first; AI then multiplies it.

10. Conclusion

This module covered the fundamentals: vocabulary, principles, and platform tour. None of these topics is the most exciting part of operating a Datadog environment — but every one of them is a precondition for what comes after. A team that hasn't internalized the difference between monitoring and observability, that hasn't agreed on SLOs, that hasn't standardized on Unified Service Tagging, that doesn't know which Datadog view to open when the page fires, will struggle with every later module. A team that has these in place can move quickly through the depth that follows.

The themes worth carrying forward into later modules:

6. Observability is the ability to ask new questions, not just check known dashboards. Instrument for the questions you can't predict.
7. Metrics, logs, traces, profiles. Four pillars, complementary. Each has the right job; use all four.
8. Define what 'reliable' means before measuring it. SLOs turn arguments into math; error budgets turn risk into a quantitative decision.
9. Tag everything consistently. Unified Service Tagging unlocks the platform; inconsistent tagging hides it.
10. Cost discipline is distributed, not centralized. Cardinality awareness lives in every developer, every code review.
11. The platform is a path through views, not a single dashboard. Service Catalog, APM, Logs, Profiler, Dashboards, Notebooks — practice the path.
12. Choose tools based on what your team does best, not on feature checklists. Datadog is excellent at SaaS-native unified observability; that's the use case to evaluate against.
13. AI is a multiplier, not a replacement. Watchdog and Bits make a well-instrumented team faster; they don't substitute for instrumentation.

The remaining modules go deep on each piece: the agent's internals, log management, APM and tracing, monitors and alerting, dashboards and notebooks, security, and operations at scale. With the foundations from this module, every one of them becomes a depth-first dive into a topic you already have the vocabulary to navigate.

End of Module 1

Observability & Datadog Fundamentals · v1.0 · 2026